

SPI / I2C Peripheral Controller

Interview Preparation Guide

What I built, how I built it, how to explain it, and where to study each concept further

1. Say this first — What I did, how it worked, what the result was

This is the very first thing to say when asked about this project — before any detail, before any bug story. Interviewers form their first impression from these three beats: what the thing is, how it actually works, and whether it worked. Say it in this order, then stop and let them ask a follow-up.

What I did

"I designed a peripheral controller chip that speaks two different serial protocols — SPI and I2C — and can act as either the master or the slave on either one, with the configuration chosen at runtime through a register, not fixed at build time. I wrote it in Verilog, then verified it with a SystemVerilog testbench, then ran it through logic synthesis to see what it turns into as real gates."

How it worked

"The chip has four separate engines inside it — SPI master, SPI slave, I2C master, I2C slave — but only one is ever actually switched on at a time; a register file picks which one. To actually test it, I built two copies of my own chip and put them on the same wires, one set as master and one set as slave, so they would talk to each other for real, the same way you'd validate a chip against an actual external part rather than just testing it against itself."

What the result was

"The first time I ran the testbench, it wasn't clean — it failed on both protocols. I spent the verification effort tracking down five separate, real bugs: two in the SPI logic, and three in the I2C logic, each one found by reading simulation logs, and for the harder ones, by adding my own debug print statements and eventually capturing an actual waveform. After fixing all five, the full test suite — fourteen separate checks across both protocols — passes completely, both when I run it directly and through an automated regression script. I also synthesized the design and confirmed it maps to about 1,073 gates."

That's the whole story in three beats. Everything past this point in the guide — the glossary, the architecture walkthrough, the five bugs in detail — is there for when they ask "tell me more" or "walk me through one of those bugs." Don't front-load those; let the three beats above stand on their own first.

Condensed, one-breath version (if they want it even shorter)

"I designed a dual-protocol SPI/I2C controller that can be master or slave on either bus at runtime, verified it with a SystemVerilog testbench that puts two instances of my own design on the same bus to talk to each other, found and fixed five real bugs using simulation logs and one waveform capture, and finished with all fourteen test checks passing and a synthesized gate count of about 1,073 cells."

2. Concepts you must be able to explain cold

These are the terms that come up almost immediately in any follow-up question about this project. Know each of these well enough to explain to someone who has never heard of them.

SPI (Serial Peripheral Interface)

A synchronous, full-duplex, 4-wire protocol: SCLK (clock, driven by the master), MOSI (master-out, slave-in), MISO (master-in, slave-out), and CS_N (chip select, active-low, one per slave). Data moves in both directions on the same clock edges — that's what "full-duplex" means here: the master doesn't just send and then separately receive, both happen simultaneously bit-by-bit.

CPOL and CPHA (the "SPI modes")

CPOL sets the clock's idle level (0 = idle low, 1 = idle high). CPHA sets which clock edge data is sampled on relative to which edge it's driven on. Together they give 4 modes (00, 01, 10, 11). The practical rule I'd say out loud: "CPHA=0 means the first bit is already sitting on the line before the first clock edge — driven during chip-select assertion — and every subsequent bit is driven on the trailing edge and sampled on the leading edge. CPHA=1 flips that: drive on leading, sample on trailing." This is exactly the distinction that caused bug 7.1 in my report — I had those two cases swapped in one line of code.

I2C (Inter-Integrated Circuit)

A synchronous, half-duplex, 2-wire bus: SCL (clock) and SDA (data), both open-drain — meaning any device can only pull the line low; an external pull-up resistor brings it high when nobody is pulling it down. This is what lets multiple

devices share the same two wires without damaging each other. A transaction is: START condition, 7-bit address + 1 read/write bit, ACK/NACK, one or more data bytes each followed by ACK/NACK, STOP condition.

START and STOP conditions

Defined entirely in terms of SDA relative to SCL: START = SDA falls while SCL is high. STOP = SDA rises while SCL is high. Critically, this means any unintended SDA transition while SCL happens to be high looks exactly like a STOP to every device on the bus — this is the exact mechanism behind two of the five bugs I found (7.4 and 7.5).

Open-drain / wired-AND

A device can only actively drive a line to 0; it can never actively drive a 1, only release the line (go to a non-driving, high-impedance state) and let the external pull-up resistor bring it high. If any device on the bus is pulling low, the whole bus reads low — hence "wired-AND." This is why my RTL models these pins as separate `oe` (output-enable) and `in` (sensed value) signals rather than a single driven wire.

Clock domain crossing (CDC) and synchronizers

Any signal that changes based on a different clock (or no clock at all, like an external bus) is asynchronous to your logic. Sampling it directly with a flip-flop risks metastability — the flip-flop's output briefly sitting at neither a valid 0 nor 1 if the input changed too close to the clock edge. The standard fix is a synchronizer: two (sometimes three) flip-flops in series, so any metastable event has a full clock cycle to resolve before it reaches the rest of your logic. My `spi_slave` and `i2c_slave` modules both do this on every external pin.

Clock gating

A power-saving technique: stop toggling the clock input to a block of logic when that logic has nothing to do, since a clock that isn't toggling can't cause any switching activity (which is where dynamic power goes). The subtlety I actually learned the hard way (bug 7.3): you cannot gate the clock of anything that has to keep passively watching an external, asynchronous signal, because gating freezes its synchronizer mid-flight, and it wakes up with a stale, wrong view of the world.

Metastability

The physical phenomenon underlying CDC bugs: a flip-flop samples its input right as that input is changing, and its output takes an unpredictable, possibly very long time to settle to a valid logic level, potentially propagating an invalid value into your design. It's not something you can see in RTL simulation — simulators model signals as clean 0/1/X, not physical voltage settling — which is exactly why CDC bugs are notoriously invisible in simulation and need dedicated structural tools (or real hardware) to catch.

ACK / NACK

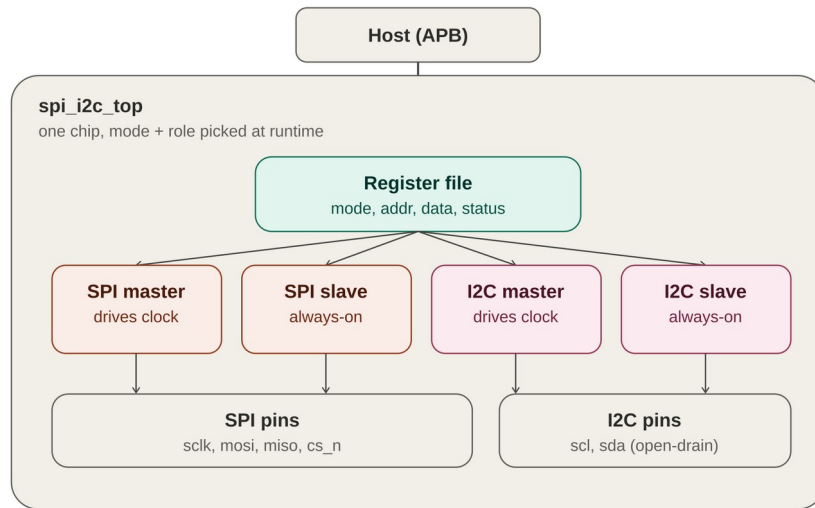
After every byte on I2C, the receiver pulls SDA low for one bit period to acknowledge (ACK). If it leaves SDA high instead, that's a NACK — "I didn't get that" or "I don't want any more data," depending on context (e.g. a master NACKs the last byte it wants to read, to tell the slave to stop sending).

Arbitration (I2C, multi-master)

If two masters start a transaction at the same time, each monitors the bus while it thinks it's driving; if a master ever sees the bus at a different value than what it itself is driving, another master won — it backs off. My design implements a simplified version of this check, restricted to phases where the master itself is actively transmitting a bit, since I found (bug 7.3's sibling issue) that checking it during ACK-release phases produces false positives, because a slave pulling the line low to ACK looks identical to "someone else is driving the bus."

3. Architecture — how to draw this on a whiteboard

If asked to sketch the design, draw this, in this order: a register file in the middle; four boxes around it labeled SPI master, SPI slave, I2C master, I2C slave; one arrow from the register file into each box (control signals); and two shared pin groups at the bottom — SPI pins (shared by the two SPI boxes) and I2C pins (shared by the two I2C boxes). Say out loud: "Only one of the four boxes is ever actually active — the register file's mode and role bits pick exactly one."



The actual diagram, for reference.

The one design decision worth mentioning unprompted: clock gating is applied only to the two master engines, not the slaves. I'll explain why if asked — it's literally bug 7.3 below, and bringing it up yourself signals that you understand the "why" behind your own architecture, not just the "what."

4. How each module actually works (implementation notes)

clk_gate.v

A latch, clocked on the falling edge of the input clock, that re-times the enable signal. The gated clock output is simply (input clock) AND (latched enable). The reason for the latch instead of just AND-ing the raw enable directly: if enable changed right as the clock was about to rise, a plain AND could produce a runt pulse (a clock edge much shorter than a real one) on the gated output, which downstream flip-flops might or might not register — a real hazard. Re-timing enable on the falling edge guarantees it's stable well before the next rising edge.

spi_master.v and spi_slave.v

Both use two separate 8-bit shift registers per transfer — one purely for the byte being transmitted, one purely for the byte being received — instead of one shared register. I chose this because SPI is genuinely full-duplex: the bit going out and the bit coming in at the same moment are unrelated data, and trying to reuse one register for both directions is exactly the kind of "clever" simplification that creates subtle aliasing bugs (which, not coincidentally, is close to what went wrong in bugs 7.1 and 7.2 — both were about exactly which edge each register updates on).

The master generates SCLK from a free-running divider counter; the slave never generates a clock — it only reacts to edges it detects on the incoming SCLK line through its synchronizer, which is why the slave module is structurally the more delicate of the two: everything it does is timed relative to a signal it didn't create.

i2c_master.v and i2c_slave.v

These are considerably more stateful than the SPI engines because I2C has more distinct phases per transaction (START, address, address-ACK, data, data-ACK, STOP) and, on top of that, roles reverse mid-transaction on a read (the slave becomes the one driving data). I implemented this as an explicit state machine with a phase field to know which of the several "byte + ACK" segments is currently active, since address bits, write-data bits, and read-data bits all share almost the same shifting logic but differ in who is driving.

The single hardest lesson from this module: a state transition that happens "as soon as I see the rising edge of the clock" is not the same moment as "as soon as my peer has finished using this bit period." Two of my five bugs (7.4, 7.5) were exactly this mismatch between when the slave thought a bit was over and when the master actually finished checking it.

spi_i2c_regs.v

A conventional memory-mapped register file: writes are simple, one register per address; status bits are "sticky" (they latch high on an event and stay high until explicitly cleared) because the CPU/testbench reading them might not poll fast enough to catch a single-cycle pulse otherwise. Clearing is write-1-to-clear (W1C), a standard pattern that avoids accidentally clearing a flag by writing back a stale read-modify-write value.

spi_i2c_top.v

Pure structural glue: instantiates everything above, decodes {mode, role} into four mutually-exclusive select signals, routes each select signal to the appropriate clock gate (masters only) or directly to the always-on clock (slaves), and multiplexes each engine's pins onto the shared physical SPI/I2C pin sets using continuous assignments that drive high-impedance (1'bz) when that engine isn't the active one.

5. How I verified it — and why I built the testbench this way

Why two instances of my own design, not a simple loopback

I could have wired one SPI master's MOSI straight back to its own MISO and called it "tested." I didn't, because that only proves the shift register can shift — it says nothing about whether my master and my slave actually agree on protocol timing with each other. Instantiating the design twice, once as master and once as slave, on a shared bus, is much closer to how you'd validate real silicon against a real external device.

The self-checking scoreboard

Every directed test issues a transaction, waits for completion by polling a status register (with a timeout, so a hung design fails loudly instead of hanging the simulation forever), then compares the actual received byte and status flags against the expected values, incrementing a pass or fail counter and printing a labeled PASS/FAIL line. This is the simplest form of self-checking testbench, and it's enough for a design this size; larger designs typically move to a class-based UVM scoreboard for reuse across many more tests.

What assertions and coverage added (and their real limitation here)

I wrote SystemVerilog Assertions for protocol invariants (e.g., "SCLK never toggles while chip-select is deasserted") and functional coverage bins (all 4 SPI modes exercised, several I2C transaction outcomes, corner-case byte values). The honest caveat, worth stating proactively in an interview: Icarus Verilog, the free simulator I used for day-to-day iteration, doesn't implement concurrent assertions or covergroups at all, so those checks are written to run under a simulator that supports the full SystemVerilog Assertion standard, while my actual day-to-day regression relied on the scoreboard. Saying this clearly, unprompted, is better than hoping nobody asks — it shows you know the difference between "wrote it" and "it's actually running in my regression."

6. The five bugs, in interview-answer (STAR) format

These are written the way you'd actually say them out loud, not the way they're written in the formal report. Practice saying these; don't read them.

Bug 1 — SPI master, swapped CPHA branch

Situation/Task: In verification, the SPI master was capturing the wrong byte from the slave in 3 of 4 CPOL/CPHA modes — always the correct value shifted by one bit. The 4th mode passed, which made it look fine at first.

Action: I noticed the one passing mode happened to use all-zero test data, which would hide a shift-by-one bug completely. That told me not to trust that pass. I traced the final-bit-capture logic and found a ternary expression with the CPHA=0 and CPHA=1 branches literally swapped.

Result: Swapped the branches; all 4 modes passed with real, non-trivial data. What I took from it: a passing test is only meaningful if you know your test data could actually reveal the bug — otherwise a pass tells you nothing.

Bug 2 — SPI slave, missing first-edge guard

Situation/Task: After fixing bug 1, CPHA=1 modes still failed, but only in the slave-to-master direction.

Action: I compared the master's and slave's shifting logic side by side. The master had an explicit guard to not shift its output register on the very first clock edge of a byte, so the pre-loaded first bit stays put long enough to be sampled. The slave was missing that exact guard.

Result: Added the same guard to the slave. Lesson: when two sides of a protocol implement mirrored logic, any asymmetry between them is a bug until proven otherwise — I should have written that guard once and shared it, in hindsight.

Bug 3 — Clock-gated slave engines, stale synchronizer state

Situation/Task: I2C tests started failing intermittently — same code, different results depending on what ran before it, in a design with zero randomness.

Action: Intermittent failure in deterministic RTL means state is leaking between tests. I traced it to my own clock-gating: I'd gated the slave engines' clocks whenever they were deselected, which froze their input synchronizers mid-flight. When re-enabled, they resumed from a stale bus reading and could misread the first real edge of the next transaction.

Result: Restricted clock gating to the two master engines only; slaves now run on the always-on clock. I now describe this, correctly, as a clock-domain-crossing bug — not just a "power logic" bug — because the actual failure mode was a synchronizer losing sync with reality.

Bug 4 — I2C slave, SDA released mid-ACK-phase

Situation/Task: A write to a matching, correctly-detected address was unexpectedly NACKed, and the slave never registered receiving the byte.

Action: I added debug prints tracing both chips' state machines cycle by cycle. Found that the slave's next state released SDA unconditionally on its very first active cycle — which was still mid-way through the previous ACK bit's high phase. By the I2C spec, SDA rising while SCL is high is a STOP condition, indistinguishable from a real one, so the transaction was silently killed.

Result: Made the release conditional on SCL actually falling first. This is the bug I'd lead with if asked "tell me about a hard bug" — it's subtle, it's a real protocol-timing hazard, and the fix is small and clean once found.

Bug 5 — I2C slave, ACK released one phase early at transaction end

Situation/Task: After fixing bug 4, the transaction completed correctly end-to-end — right byte, right rx_valid flag — but the master's own status register still showed an ACK error.

Action: Captured an actual waveform (not just logs, this time) and found the slave released its ACK line about 8 clock cycles before the master ever checked it — the slave reacted to the rising edge of SCL immediately, while the master's own high-phase counter ran for several more cycles before it actually sampled.

Result: Added a holding state that keeps the ACK line driven until the master genuinely pulls SCL low again for whatever comes next. The generalizable lesson: in a shared-bus, multi-agent design, "when I'm done" and "when my peer checks" are two separate assumptions that must be explicitly reconciled, not left implicit.

7. Likely interview questions and model answers

Grouped roughly from "about this project specifically" to "general digital design/verification fundamentals this project demonstrates."

Q: Why did you build two instances of your own design instead of a simpler testbench?

A: Because a self-loopback only proves my shift logic works, not that my master and slave agree with each other on protocol timing. Two independent instances on a shared bus is a much stronger test of real interoperability, and it's exactly how you'd validate the IP against a real external part.

Q: Why is clock gating only applied to the master engines?

A: Because the slave engines contain synchronizers that must continuously track an external, asynchronous bus. Gating their clock freezes that synchronizer mid-flight; when re-enabled it resumes from a stale bus reading. I found this the hard way — it caused an intermittent, run-order-dependent test failure — and the fix was architectural: only gate logic that owns its own timeline.

Q: What is the difference between simulation and formal verification, and would formal have helped here?

A: Simulation runs specific test vectors and checks specific outcomes; it only finds bugs your tests happen to exercise. Formal verification mathematically proves a property holds for every possible input sequence. My two I2C ACK-timing bugs were both found by directed simulation that happened to hit the right timing window — a formal tool given the property "SDA never rises while SCL is high except at a legitimate START/STOP" would have found the same bug immediately, without needing the right test data by luck.

Q: What is the difference between functional coverage and code coverage?

A: Code coverage measures whether your simulation exercised lines/branches/toggles of the RTL itself — it tells you what code ran, not whether you tested anything meaningful. Functional coverage is something you explicitly define based on the specification (e.g., "did I exercise all 4 SPI modes?") — it tells you whether your test plan is actually complete relative to what the design is supposed to do.

Q: Why did you use two separate shift registers in the SPI engines instead of one?

A: SPI is full-duplex — the bit going out and the bit coming in at the same instant are unrelated data. Reusing one register for both would require very careful timing to avoid aliasing transmitted and received bits into the same storage, which is exactly the class of subtlety that caused two of my five bugs elsewhere in this project. Two registers is simpler to reason about and, in this design, cheap.

Q: What is metastability and how do you protect against it?

A: It's what happens when a flip-flop samples an input that's changing right at the clock edge — its output can take an unbounded time to settle to a valid 0 or 1. The standard mitigation is a synchronizer: 2 (or 3) flip-flops in series on the receiving clock, giving any metastable event a full cycle to resolve before it reaches downstream logic. Every external pin in my spi_slave and i2c_slave modules goes through one.

Q: What would a structural CDC tool (like Conformal CDC) have told you about this design?

A: It almost certainly would have flagged my clock-gating bug (bug 3) automatically and immediately, without running any simulation — gating the clock of logic that synchronizes an asynchronous input is a textbook CDC hazard pattern that these tools specifically check for.

Q: Walk me through what happens between RTL and a synthesized netlist.

A: The synthesis tool reads the RTL, elaborates the module hierarchy, and maps the described behavior onto library cells — in my case, generic gate primitives since I didn't have a real process library available. It performs optimizations (constant propagation, common sub-expression elimination, technology mapping) and outputs a structural netlist plus reports like cell count and area. What it does not do is verify functional correctness — that's why I ran my testbench, unmodified, against the synthesized netlist conceptually as a follow-up step, though I didn't gate-level-simulate it in this project.

Q: What is static timing analysis and why didn't you get real numbers?

A: STA checks that every signal path settles before the next clock edge is allowed to depend on it, across setup and hold constraints, without needing to simulate every possible input. It needs a real timing library (a .lib file) with actual gate delays, which comes from a foundry PDK; I didn't have one available, so I prepared the OpenSTA script and constraints file but didn't get real numbers out of it — I say this directly rather than implying I ran it.

Q: What's the difference between an SPI slave with clock stretching and I2C clock stretching?

A: SPI has no clock-stretching mechanism at all — the master fully owns SCLK timing and the slave must keep up. I2C explicitly allows a slave to hold SCL low to pause the master, which my i2c_master implements (it waits for SCL to actually read high before starting its high-phase count) — but my i2c_slave doesn't implement stretching itself, which I'd note as a known limitation if asked what I'd add next.

Q: If you had to add UVM to this project, what would change?

A: I'd replace the directed test tasks with randomized sequences (random CPOL/CPHA/data/address combinations) driven through a UVM driver/sequencer, add a proper scoreboard component instead of inline checks, and let functional coverage, collected through real covergroups, tell me when I'd exercised enough of the space — rather than me hand-picking a fixed list of directed cases like I did here.

8. Where to study each topic further

Verilog / SystemVerilog language fundamentals

- **Verilog tutorial** — [ChipVerify — Verilog](#)
- **SystemVerilog tutorial** — [ChipVerify — SystemVerilog](#)
- **General reference** — [VLSIVerify](#)

Verification methodology, assertions, coverage

- **SVA and assertions** — [Siemens Verification Academy — Assertions](#)
- **UVM** — [ChipVerify — UVM](#)

Formal verification

- **Formal verification overview** — [Siemens Verification Academy — Formal Verification](#)
- **Practical, code-first formal verification using free tools** — [The ZipCPU blog \(formal + CDC series\)](#)

Clock domain crossing and synchronizers — directly relevant to bug 7.3

- **CDC fundamentals** — [ZipCPU — Some Simple Clock-Domain Crossing Solutions](#)

- **Async FIFO + formally verifying it** — [ZipCPU — Crossing Clock Domains with an Asynchronous FIFO](#)

Protocol specifications

- **I2C-bus specification and user manual (UM10204)** — [NXP official spec \(may require a free NXP account\)](#)
- **I2C overview / mirror if the NXP link requires login** — [i2c-bus.org](#)

Synthesis and open-source EDA

- **Yosys documentation** — [YosysHQ](#)
- **SkyWater sky130 open PDK (for real synthesis/STA numbers)** — [GitHub](#) — [google/skywater-pdk](#)

This project's own files, for reference while studying

The full source is reproduced in Appendix A below, so you can study the actual code without needing the original project files open.

Appendix A: Complete source code

Every RTL and testbench file exactly as it exists in the final, passing version of the project (14/14 checks passing). Presented in the order you'd want to read them: smallest/simplest module first, top-level integration last, testbench files after that.

A.1 clk_gate.v

```
//=====
// clk_gate.v
// Latch-based clock gating cell (standard "ICG" RTL idiom).
//
// Gates clk_in with `enable`, using a negedge-triggered latch to re-time the
// enable so the gated clock cannot glitch when `enable` changes near a rising
// edge of clk_in. `test_en` forces the clock through during scan/ATPG.
//
// In a real ASIC flow this RTL pattern is what synthesis tools (DC, Genus,
// Yosys+techmap) recognize and map onto a technology clock-gating cell
// (e.g. CKLNQD*, or CLKGATE_X1 in an open PDK) -- do NOT rely on the raw
// latch+AND surviving synthesis untouched; that mapping is what actually
// saves power. See syn/yosys_synth.tcl for how this is handled.
//=====
module clk_gate (
    input wire clk_in,
    input wire enable,    // level enable: 1 = clock passes through
    input wire test_en,   // scan/DFT override: forces clock always-on
    output wire clk_out
);

    reg en_latched;

    always @(negedge clk_in) begin
        if (test_en)
            en_latched <= 1'b1;
        else
            en_latched <= enable;
    end

    assign clk_out = clk_in & en_latched;

endmodule
```

A.2 spi_master.v

```
//=====
// spi_master.v
// Synthesizable SPI master engine.
//
// - Supports all 4 SPI modes via {cpol,cpha}.
// - SCLK frequency = clk / (2*(clkdiv+1)).
// - Single in-flight transfer of DW bits, MSB first, full-duplex.
// - Uses two independent shift registers (tx_shreg drives MOSI, rx_shreg
//   captures MISO) so the drive-edge and sample-edge can differ per CPHA
//   without aliasing TX/RX data in one register.
//
// Protocol timing reference (SPI mode terminology):
//   CPHA=0: data driven a half period *before* the first SCLK edge
//            (during CS assertion), sampled on the leading edge, next bit
//            driven on the trailing edge.
//   CPHA=1: data driven on the leading edge, sampled on the trailing edge.
//   CPOL selects the idle level of SCLK (0 = idle low, 1 = idle high).
//=====
module spi_master #(
    parameter DW = 8
) (
    input wire          clk,
    input wire          rst_n,

    input wire          en,          // block enable; must be 1 to start
```

```

input wire      start,      // 1-cycle pulse: begin a transfer
input wire      cpol,
input wire      cpha,
input wire [15:0] clkdiv,   // SCLK half-period in clk cycles - 1

input wire [DW-1:0] tx_data,
output reg [DW-1:0] rx_data,
output reg      busy,
output reg      done,      // 1-cycle pulse on transfer completion

output reg      sclk,
output wire     mosi,
input wire      miso,
output reg      cs_n
);

localparam ST_IDLE   = 3'd0;
localparam ST_CS_SETUP = 3'd1;
localparam ST_RUN    = 3'd2;
localparam ST_CS_HOLD = 3'd3;

localparam EDGE_W = 8; // supports DW up to 127
localparam EDGE_MAX = (2*DW) - 1;

reg [2:0]      state;
reg [15:0]     div_cnt;
reg [EDGE_W-1:0] edge_idx;
reg [DW-1:0]   tx_shreg;
reg [DW-1:0]   rx_shreg;

wire div_tick = (div_cnt == clkdiv);
wire is_leading_edge = ~edge_idx[0]; // even index = leading edge

assign mosi = tx_shreg[DW-1];

// clock divider counter - runs whenever a transfer is in flight
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        div_cnt <= 16'd0;
    else if (state == ST_IDLE)
        div_cnt <= 16'd0;
    else if (div_tick)
        div_cnt <= 16'd0;
    else
        div_cnt <= div_cnt + 16'd1;
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= ST_IDLE;
        sclk <= 1'b0;
        cs_n <= 1'b1;
        busy <= 1'b0;
        done <= 1'b0;
        edge_idx <= {EDGE_W{1'b0}};
        tx_shreg <= {DW{1'b0}};
        rx_shreg <= {DW{1'b0}};
        rx_data <= {DW{1'b0}};
    end else begin
        done <= 1'b0;

        case (state)
            //-----
            ST_IDLE: begin
                sclk <= cpol;
                cs_n <= 1'b1;
                busy <= 1'b0;
                if (en && start) begin
                    tx_shreg <= tx_data; // CPHA0: MSB now valid on MOSI
                    cs_n <= 1'b0;
                    busy <= 1'b1;

```

```

        edge_idx <= {EDGE_W{1'b0}};
        state <= ST_CS_SETUP;
    end
end

//-----
ST_CS_SETUP: begin
    // hold CS asserted for one divider period before the
    // first SCLK edge (satisfies CS-to-SCLK setup time and
    // gives CPHA=0 slaves time to see the pre-driven bit)
    if (div_tick)
        state <= ST_RUN;
    end

//-----
ST_RUN: begin
    if (div_tick) begin
        sclk <= ~sclk;

        if (is_leading_edge) begin
            if (!cpha)
                rx_shreg <= {rx_shreg[DW-2:0], miso}; // sample
            else if (edge_idx != 0)
                tx_shreg <= {tx_shreg[DW-2:0], 1'b0}; // drive next
        end else begin
            if (!cpha)
                tx_shreg <= {tx_shreg[DW-2:0], 1'b0}; // drive next
            else
                rx_shreg <= {rx_shreg[DW-2:0], miso}; // sample
        end

        if (edge_idx == EDGE_MAX) begin
            state <= ST_CS_HOLD;
            // CPHA=0 already sampled the last bit on the
            // previous leading edge, so rx_shreg is already
            // complete here - use it as-is. CPHA=1 samples
            // its last bit on this very (trailing) edge, so
            // rx_shreg is still one cycle stale here and we
            // must build the final value fresh from miso.
            rx_data <= (!cpha) ? rx_shreg
                        : {rx_shreg[DW-2:0], miso};
        end else begin
            edge_idx <= edge_idx + 1'b1;
        end
    end
end

//-----
ST_CS_HOLD: begin
    if (div_tick) begin
        cs_n <= 1'b1;
        busy <= 1'b0;
        done <= 1'b1;
        state <= ST_IDLE;
    end
end

default: state <= ST_IDLE;
endcase
end
end
endmodule

```

A.3 spi_slave.v

```

//=====
// spi_slave.v
// Synthesizable SPI slave engine.

```

```

//
// SCLK/MOSI/CS_N are external, asynchronous to `clk`, so they are brought in
// through 2-FF synchronizers and edge-detected in the system clock domain.
// This assumes clk oversamples SCLK by a healthy margin (>=4x recommended) -
// a standard, documented constraint for synchronous-fabric SPI slaves.
//
// Supports all 4 SPI modes via {cpol,cpha}; multiple back-to-back bytes are
// supported while CS_N stays asserted (tx_data is re-latched after each byte).
//=====
module spi_slave #(
    parameter DW = 8
) (
    input wire          clk,
    input wire          rst_n,
    input wire          en,

    input wire          cpol,
    input wire          cpha,

    input wire          sclk_in,
    input wire          mosi_in,
    input wire          cs_n_in,
    output wire         miso_out,

    input wire [DW-1:0] tx_data,    // next byte to shift out
    output reg [DW-1:0] rx_data,
    output reg          rx_valid,    // 1-cycle pulse: full byte received
    output reg          busy
);

// ----- input synchronizers -----
reg [1:0] sclk_sync, cs_sync, mosi_sync;
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        sclk_sync <= 2'b00;
        cs_sync   <= 2'b11;
        mosi_sync <= 2'b00;
    end else begin
        sclk_sync <= {sclk_sync[0], sclk_in};
        cs_sync   <= {cs_sync[0],  cs_n_in};
        mosi_sync <= {mosi_sync[0], mosi_in};
    end
end

wire sclk_s = sclk_sync[1];
wire cs_n_s = cs_sync[1];
wire mosi_s = mosi_sync[1];

reg sclk_prev, cs_n_prev;
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        sclk_prev <= 1'b0;
        cs_n_prev <= 1'b1;
    end else begin
        sclk_prev <= sclk_s;
        cs_n_prev <= cs_n_s;
    end
end

wire sclk_rise = ~sclk_prev & sclk_s;
wire sclk_fall = sclk_prev & ~sclk_s;

wire leading_edge = cpol ? sclk_fall : sclk_rise;
wire trailing_edge = cpol ? sclk_rise : sclk_fall;

wire cs_falling = cs_n_prev & ~cs_n_s; // start of transaction
wire cs_rising  = ~cs_n_prev & cs_n_s; // end of transaction

// ----- shift engine -----
reg [DW-1:0] tx_shreg;
reg [DW-1:0] rx_shreg;

```

```

reg [3:0]    bit_cnt;

assign miso_out = tx_shreg[DW-1];

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        tx_shreg <= {DW{1'b0}};
        rx_shreg <= {DW{1'b0}};
        rx_data  <= {DW{1'b0}};
        bit_cnt  <= 4'd0;
        rx_valid <= 1'b0;
        busy     <= 1'b0;
    end else begin
        rx_valid <= 1'b0;

        if (!en) begin
            busy <= 1'b0;
        end else if (cs_falling) begin
            tx_shreg <= tx_data; // CPHA0: MSB pre-driven for master to sample
            bit_cnt  <= 4'd0;
            busy     <= 1'b1;
        end else if (cs_rising) begin
            busy <= 1'b0;
        end else if (busy) begin
            if (leading_edge) begin
                if (!cpha)
                    rx_shreg <= {rx_shreg[DW-2:0], mosi_s}; // sample
                else if (bit_cnt != 4'd0)
                    tx_shreg <= {tx_shreg[DW-2:0], 1'b0}; // drive next (skip on the
                                                            // first leading edge of a
                                                            // byte - the pre-loaded MSB
                                                            // must stay put for it)
            end else if (trailing_edge) begin
                if (!cpha) begin
                    tx_shreg <= {tx_shreg[DW-2:0], 1'b0}; // drive next
                end else begin
                    rx_shreg <= {rx_shreg[DW-2:0], mosi_s}; // sample
                end

                if (bit_cnt == DW-1) begin
                    // byte complete: CPHA1 samples its final bit this
                    // very edge, so build rx_data from the fresh value;
                    // CPHA0 already holds the settled byte in rx_shreg.
                    rx_data  <= cpha ? {rx_shreg[DW-2:0], mosi_s} : rx_shreg;
                    rx_valid <= 1'b1;
                    bit_cnt  <= 4'd0;
                    tx_shreg <= tx_data; // reload for next byte, CS still low
                end else begin
                    bit_cnt <= bit_cnt + 4'd1;
                end
            end
        end
    end
end
end
end
end
endmodule

```

A.4 i2c_master.v

```

//=====
// i2c_master.v
// Synthesizable I2C master engine (single-byte transactions, 7-bit addressing).
//
// Bus model: open-drain, modeled as separate output-enable (`*_oe`, 1 = pull
// line low) and input (`*_in`) per wire; an external pull-up (or the
// testbench's pull-up model) brings the line high whenever oe=0 on every
// driver. The actual tri-state pad belongs at the top level / IO ring, not
// in this core.
//

```

```

// Sequence: START -> 7-bit addr + R/W -> slave ACK -> one data byte
// (direction per R/W) -> ACK (write: slave drives / read: master drives,
// always NACK since this is a single-byte transfer) -> STOP.
//
// Features:
// - Clock stretching: after releasing SCL, the master waits for
//   scl_in==1 before starting the high-phase timer, so a slow slave can
//   hold SCL low to gain time.
// - Simple arbitration-loss detection: whenever the master releases SDA
//   expecting it to float high, if sda_in reads back 0 while SCL is high,
//   another device is driving the bus -> abort, flag arb_lost.
//
// clkdiv sets each SCL half-period in clk cycles (SCL period is
// approximately 2*(clkdiv+1) clk cycles, plus any clock-stretch delay).
//=====
module i2c_master #(
    parameter AW = 7
) (
    input wire      clk,
    input wire      rst_n,
    input wire      en,
    input wire      start,      // 1-cycle pulse: begin a transaction
    input wire [15:0] clkdiv,

    input wire [AW-1:0] slv_addr,
    input wire      rw,        // 0 = write, 1 = read
    input wire [7:0]  wr_data,
    output reg [7:0]  rd_data,

    output reg      busy,
    output reg      done,      // 1-cycle pulse on completion
    output reg      ack_err,   // address or data byte was NACKed
    output reg      arb_lost,

    output reg      scl_oe,    // 1 = actively pull SCL low
    input wire      scl_in,
    output reg      sda_oe,    // 1 = actively pull SDA low
    input wire      sda_in
);

// ---- macro sequencing states ----
localparam S_IDLE      = 3'd0;
localparam S_START     = 3'd1;
localparam S_CLK_LOW   = 3'd2;
localparam S_CLK_RELSCL = 3'd3; // release SCL, wait out any stretching
localparam S_CLK_HIGH  = 3'd4;
localparam S_STOP_SETUP = 3'd5; // force SDA low with SCL low before STOP
localparam S_STOP      = 3'd6; // release SDA while SCL high -> STOP
localparam S_DONE      = 3'd7;

// ---- which byte/ack the S_CLK_* loop is currently clocking ----
localparam P_ADDR      = 3'd0; // master drives addr+rw bits
localparam P_ADDR_ACK  = 3'd1; // slave drives ack
localparam P_WDATA     = 3'd2; // master drives write-data bits
localparam P_WDATA_ACK = 3'd3; // slave drives ack
localparam P_RDATA     = 3'd4; // slave drives read-data bits
localparam P_RACK      = 3'd5; // master drives ack/nack (always nack here)

reg [2:0] state;
reg [2:0] phase;
reg [3:0] bit_cnt;
reg [7:0] shreg;
reg [15:0] div_cnt;

wire div_tick = (div_cnt == clkdiv);
wire timed_state = (state == S_CLK_LOW) || (state == S_CLK_HIGH) ||
                  (state == S_START)   || (state == S_STOP_SETUP) ||
                  (state == S_STOP);

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)

```

```

        div_cnt <= 16'd0;
    else if (!timed_state || div_tick)
        div_cnt <= 16'd0;
    else
        div_cnt <= div_cnt + 16'd1;
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= S_IDLE;
        phase <= P_ADDR;
        bit_cnt <= 4'd0;
        shreg <= 8'd0;
        scl_oe <= 1'b0;
        sda_oe <= 1'b0;
        busy <= 1'b0;
        done <= 1'b0;
        ack_err <= 1'b0;
        arb_lost <= 1'b0;
        rd_data <= 8'd0;
    end else begin
        done <= 1'b0;

        case (state)
            //-----
            S_IDLE: begin
                scl_oe <= 1'b0; // bus idle: both lines released high
                sda_oe <= 1'b0;
                ack_err <= 1'b0;
                arb_lost <= 1'b0;
                if (en && start) begin
                    busy <= 1'b1;
                    shreg <= {slv_addr, rw};
                    bit_cnt <= 4'd7;
                    phase <= P_ADDR;
                    state <= S_START;
                end
            end

            //-----
            S_START: begin
                // SCL assumed released/high on entry; pull SDA low
                // while SCL is still high -> START condition.
                sda_oe <= 1'b1;
                if (div_tick)
                    state <= S_CLK_LOW; // fall into bit 0 (MSB) of addr
            end

            //-----
            S_CLK_LOW: begin
                scl_oe <= 1'b1; // drive SCL low, set up SDA underneath it
                case (phase)
                    P_ADDR, P_WDATA: sda_oe <= ~shreg[7];
                    P_ADDR_ACK, P_WDATA_ACK, P_RDATA: sda_oe <= 1'b0; // release for peer
                    P_RACK: sda_oe <= 1'b0; // NACK = release (single-byte read)
                    default: sda_oe <= 1'b0;
                endcase
                if (div_tick)
                    state <= S_CLK_RELSCL;
            end

            //-----
            S_CLK_RELSCL: begin
                scl_oe <= 1'b0; // release SCL, let pull-up bring it high
                if (scl_in) // stalls here while a slave stretches
                    state <= S_CLK_HIGH;
            end

            //-----
            S_CLK_HIGH: begin
                // Arbitration only applies while the master itself is

```

```

// trying to assert a bit value (address/write-data
// phases). During ACK phases and read-data, releasing
// SDA is deliberate - it's the slave's turn to drive,
// and it pulling SDA low (e.g. a normal ACK) is
// expected, not a bus conflict.
if (!sda_oe && !sda_in && (phase == P_ADDR || phase == P_WDATA)) begin
    // expected SDA released/high but bus reads low:
    // another driver is on the bus.
    arb_lost <= 1'b1;
    state <= S_STOP_SETUP;
end else if (div_tick) begin
    state <= S_CLK_LOW; // default: continue to next bit
    case (phase)
        P_ADDR, P_WDATA: begin
            if (bit_cnt == 4'd0)
                phase <= (phase == P_ADDR) ? P_ADDR_ACK : P_WDATA_ACK;
            else begin
                bit_cnt <= bit_cnt - 4'd1;
                shreg <= {shreg[6:0], 1'b0};
            end
        end
        P_RDATA: begin
            rd_data <= {rd_data[6:0], sda_in};
            if (bit_cnt == 4'd0)
                phase <= P_RACK;
            else
                bit_cnt <= bit_cnt - 4'd1;
            end
        end
        P_ADDR_ACK: begin
            if (sda_in) begin
                ack_err <= 1'b1; // address NACKed, abort
                phase <= P_ADDR;
                state <= S_STOP_SETUP; // overrides default above
            end else if (rw) begin
                phase <= P_RDATA;
                bit_cnt <= 4'd7;
            end else begin
                phase <= P_WDATA;
                bit_cnt <= 4'd7;
                shreg <= wr_data;
            end
        end
        P_WDATA_ACK: begin
            if (sda_in) ack_err <= 1'b1; // data byte NACKed
            phase <= P_ADDR;
            state <= S_STOP_SETUP; // overrides default above
        end
        P_RACK: begin
            phase <= P_ADDR;
            state <= S_STOP_SETUP; // overrides default above
        end
        default: ;
    endcase
end
end

//-----
S_STOP_SETUP: begin
    scl_oe <= 1'b1; // ensure SCL low
    sda_oe <= 1'b1; // ensure SDA low
    if (div_tick)
        state <= S_STOP;
end

//-----
S_STOP: begin
    scl_oe <= 1'b0; // release SCL high first
    if (scl_in) begin
        if (div_tick) begin
            sda_oe <= 1'b0; // release SDA while SCL high -> STOP
            state <= S_DONE;
        end
    end
end

```

```

        end
    end
end

//-----
S_DONE: begin
    busy  <= 1'b0;
    done  <= 1'b1;
    state <= S_IDLE;
end

    default: state <= S_IDLE;
endcase
end
end
endmodule

```

A.5 i2c_slave.v

```

//=====
// i2c_slave.v
// Synthesizable I2C slave engine, 7-bit addressing, single data byte per
// transaction (matches i2c_master.v's single-byte transaction model).
//
// The slave never generates SCL (no clock stretching in this core - noted
// as a scope simplification; see docs/README.md). It reacts to SCL edges
// supplied by whichever master owns the bus:
// - samples incoming bits (address, write data, master's own ACK) on the
//   SCL rising edge,
// - drives its own outgoing bits (ACK, read data) starting at the SCL
//   falling edge so the value is settled well before the next rising edge.
//
// START = SDA falls while SCL is steady high. STOP = SDA rises while SCL is
// steady high. Either is detected combinatorially from the synchronized bus
// and takes priority over whatever phase the FSM is in, per the I2C spec.
//=====
module i2c_slave #(
    parameter AW = 7
) (
    input wire          clk,
    input wire          rst_n,
    input wire          en,
    input wire [AW-1:0] own_addr,

    input wire [7:0]    tx_data,    // byte returned when a master reads us
    output reg [7:0]    rx_data,
    output reg          rx_valid,    // 1-cycle pulse: full byte received
    output reg          busy,
    output reg          addr_match,  // 1-cycle pulse: we were addressed

    input wire          scl_in,
    output reg          sda_oe,      // 1 = actively pull SDA low
    input wire          sda_in
);

// ----- input synchronizers -----
reg [1:0] scl_sync, sda_sync;
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        scl_sync <= 2'b11;
        sda_sync <= 2'b11;
    end else begin
        scl_sync <= {scl_sync[0], scl_in};
        sda_sync <= {sda_sync[0], sda_in};
    end
end
end
wire scl_s = scl_sync[1];
wire sda_s = sda_sync[1];

```

```

reg scl_prev, sda_prev;
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        scl_prev <= 1'b1;
        sda_prev <= 1'b1;
    end else begin
        scl_prev <= scl_s;
        sda_prev <= sda_s;
    end
end

wire scl_rise = ~scl_prev & scl_s;
wire scl_fall = scl_prev & ~scl_s;
wire start_cond = scl_prev & scl_s & sda_prev & ~sda_s; // SDA falls, SCL steady high
wire stop_cond = scl_prev & scl_s & ~sda_prev & sda_s; // SDA rises, SCL steady high

// ----- main FSM -----
localparam S_IDLE = 3'd0;
localparam S_ADDR = 3'd1;
localparam S_ADDR_ACK = 3'd2;
localparam S_WDATA = 3'd3;
localparam S_WDATA_ACK = 3'd4;
localparam S_RDATA = 3'd5;
localparam S_RACK = 3'd6;
localparam S_MISMATCH = 3'd7;
localparam S_WDATA_DONE = 4'd8; // holds the ACK drive until master truly moves on

reg [3:0] state;
reg [3:0] bit_cnt;
reg [7:0] rx_shreg;
reg [7:0] tx_shreg;
reg [7:0] addr_rw_byte;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= S_IDLE;
        bit_cnt <= 4'd0;
        rx_shreg <= 8'd0;
        tx_shreg <= 8'd0;
        addr_rw_byte <= 8'd0;
        sda_oe <= 1'b0;
        busy <= 1'b0;
        rx_data <= 8'd0;
        rx_valid <= 1'b0;
        addr_match <= 1'b0;
    end else begin
        rx_valid <= 1'b0;
        addr_match <= 1'b0;

        if (stop_cond) begin
            state <= S_IDLE;
            busy <= 1'b0;
            sda_oe <= 1'b0;
        end else if (!en) begin
            state <= S_IDLE;
            busy <= 1'b0;
            sda_oe <= 1'b0;
        end else if (start_cond) begin
            state <= S_ADDR;
            bit_cnt <= 4'd7;
            busy <= 1'b1;
            sda_oe <= 1'b0;
        end else begin
            case (state)
                S_IDLE: sda_oe <= 1'b0;

                //-----
                S_ADDR: begin
                    if (scl_rise) begin
                        rx_shreg <= {rx_shreg[6:0], sda_s};

```

```

        if (bit_cnt == 4'd0) begin
            addr_rw_byte <= {rx_shreg[6:0], sda_s};
            state <= S_ADDR_ACK;
        end else begin
            bit_cnt <= bit_cnt - 4'd1;
        end
    end
end

//-----
S_ADDR_ACK: begin
    if (scl_fall)
        sda_oe <= (addr_rw_byte[7:1] == own_addr); // ACK if matched
    if (scl_rise) begin
        if (addr_rw_byte[7:1] == own_addr) begin
            addr_match <= 1'b1;
            bit_cnt <= 4'd7;
            if (addr_rw_byte[0]) begin // master wants to read us
                tx_shreg <= tx_data;
                state <= S_RDATA;
            end else begin // master will write us
                state <= S_WDATA;
            end
        end else begin
            state <= S_MISMATCH;
        end
    end
end

//-----
S_WDATA: begin
    // Only release at scl_fall (start of this bit's low
    // phase). Releasing unconditionally here would fire
    // on the very first cycle of this state - which is
    // still mid-high-phase of the preceding ACK bit -
    // and SDA falling->rising while SCL is steady high
    // looks exactly like a STOP condition on the bus.
    if (scl_fall) sda_oe <= 1'b0; // release: master drives data
    if (scl_rise) begin
        rx_shreg <= {rx_shreg[6:0], sda_s};
        if (bit_cnt == 4'd0) begin
            rx_data <= {rx_shreg[6:0], sda_s};
            state <= S_WDATA_ACK;
        end else begin
            bit_cnt <= bit_cnt - 4'd1;
        end
    end
end

//-----
S_WDATA_ACK: begin
    if (scl_fall) sda_oe <= 1'b1; // slave ACKs the byte
    if (scl_rise) begin
        rx_valid <= 1'b1;
        // Don't release SDA or go to S_IDLE yet - S_IDLE
        // releases unconditionally, and the master won't
        // actually finish sampling this ACK bit (its own
        // high-phase timer) for a while yet. Releasing
        // now would make SDA float high while the
        // master's SCL is still high - an apparent STOP.
        // Keep driving until the master genuinely pulls
        // SCL low again for whatever comes next.
        state <= S_WDATA_DONE;
    end
end

//-----
S_WDATA_DONE: begin
    if (scl_fall) begin
        sda_oe <= 1'b0;
        state <= S_IDLE;
    end
end

```

```

        end
    end

    //-----
    S_RDATA: begin
        if (scl_fall) sda_oe <= ~tx_shreg[7];
        if (scl_rise) begin
            if (bit_cnt == 4'd0) begin
                state <= S_RACK;
            end else begin
                bit_cnt <= bit_cnt - 4'd1;
                tx_shreg <= {tx_shreg[6:0], 1'b0};
            end
        end
    end
end

//-----
S_RACK: begin
    // same reasoning as S_WDATA above: only release at
    // scl_fall, not unconditionally on state entry.
    if (scl_fall) sda_oe <= 1'b0; // release: master drives ACK/NACK
    if (scl_rise)
        state <= S_IDLE; // master's ack/nack sampled by testbench monitor
end

//-----
S_MISMATCH: begin
    sda_oe <= 1'b0; // not addressed - stay off the bus
end

default: state <= S_IDLE;
endcase
end
end
end
endmodule

```

A.6 spi_i2c_regs.v

```

//=====
// spi_i2c_regs.v
// APB-lite register file / programmer's interface for the dual-mode
// SPI+I2C controller. Zero wait-state slave (pready is tied high).
//
// Address map (word-aligned, byte offsets):
// 0x00 CTRL   [0]=enable [1]=mode(0=SPI,1=I2C) [2]=role(0=MASTER,1=SLAVE)
//             [3]=cpol   [4]=cpha
// 0x04 CLKDIV [15:0] engine clock divider
// 0x08 TXDATA [7:0]  byte to transmit next
// 0x0C RXDATA [7:0]  last byte received (read-to-clear rx_valid)
// 0x10 ADDR   [6:0]  I2C target address (master) / own address (slave)
// 0x14 CMD    [0]=start (self-clearing write strobe) [1]=rw (0=wr,1=rd)
// 0x18 STATUS [0]=busy [1]=done(sticky, W1C) [2]=ack_err(W1C)
//             [3]=arb_lost(W1C) [4]=rx_valid(sticky, cleared by RXDATA rd)
// 0x1C IRQEN  [0]=done_ie (irq = done_ie & status.done)
//=====
module spi_i2c_regs (
    input wire      clk,
    input wire      rst_n,

    // APB-lite
    input wire      psel,
    input wire      penable,
    input wire      pwrite,
    input wire [7:0] paddr,
    input wire [31:0] pwrdata,
    output reg [31:0] prdata,
    output wire      pready,

```

```

// config outputs -> engines
output reg      enable,
output reg      mode,      // 0=SPI, 1=I2C
output reg      role,      // 0=MASTER, 1=SLAVE
output reg      cpol,
output reg      cpha,
output reg [15:0] clkdiv,
output reg [7:0] tx_data,
output reg [6:0] i2c_addr,
output reg      rw,
output wire      start_pulse,

// status inputs <- engines
input wire      busy_i,
input wire      done_i,      // 1-cycle pulse
input wire [7:0] rx_data_i,
input wire      rx_valid_i, // 1-cycle pulse
input wire      ack_err_i, // 1-cycle pulse
input wire      arb_lost_i, // 1-cycle pulse

output wire      irq
);

localparam ADDR_CTRL    = 8'h00;
localparam ADDR_CLKDIV  = 8'h04;
localparam ADDR_TXDATA  = 8'h08;
localparam ADDR_RXDATA  = 8'h0C;
localparam ADDR_ADDR    = 8'h10;
localparam ADDR_CMD     = 8'h14;
localparam ADDR_STATUS  = 8'h18;
localparam ADDR_IRQEN   = 8'h1C;

assign pready = 1'b1; // zero-wait-state slave

wire apb_write = psel & penable & pwrite;
wire apb_read  = psel & penable & ~pwrite;

reg      start_reg;
assign start_pulse = start_reg;

reg [7:0] rx_data_reg;
reg      sticky_done, sticky_ack_err, sticky_arb_lost, sticky_rx_valid;
reg      irq_en;

// ----- writes -----
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        enable    <= 1'b0;
        mode      <= 1'b0;
        role      <= 1'b0;
        cpol      <= 1'b0;
        cpha      <= 1'b0;
        clkdiv    <= 16'd4;
        tx_data    <= 8'd0;
        i2c_addr   <= 7'd0;
        rw        <= 1'b0;
        start_reg  <= 1'b0;
        irq_en     <= 1'b0;
    end else begin
        start_reg <= 1'b0; // self-clearing strobe

        if (apb_write) begin
            case (paddr)
                ADDR_CTRL: begin
                    enable <= pwrdata[0];
                    mode   <= pwrdata[1];
                    role   <= pwrdata[2];
                    cpol   <= pwrdata[3];
                    cpha   <= pwrdata[4];
                end
            end
        end
    end
end

```

```

        ADDR_CLKDIV: clkdiv    <= pwdata[15:0];
        ADDR_TXDATA: tx_data   <= pwdata[7:0];
        ADDR_ADDR:  i2c_addr   <= pwdata[6:0];
        ADDR_CMD: begin
            rw          <= pwdata[1];
            start_reg    <= pwdata[0];
        end
        ADDR_IRQEN: irq_en <= pwdata[0];
        default: ;
    endcase
end
end
end

// ----- sticky status -----
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        sticky_done    <= 1'b0;
        sticky_ack_err  <= 1'b0;
        sticky_arb_lost <= 1'b0;
        sticky_rx_valid <= 1'b0;
        rx_data_reg     <= 8'd0;
    end else begin
        if (done_i)      sticky_done    <= 1'b1;
        if (ack_err_i)   sticky_ack_err  <= 1'b1;
        if (arb_lost_i)  sticky_arb_lost <= 1'b1;
        if (rx_valid_i) begin
            sticky_rx_valid <= 1'b1;
            rx_data_reg     <= rx_data_i;
        end
    end

    // write-1-to-clear on STATUS
    if (apb_write && paddr == ADDR_STATUS) begin
        if (pwdata[1]) sticky_done    <= 1'b0;
        if (pwdata[2]) sticky_ack_err  <= 1'b0;
        if (pwdata[3]) sticky_arb_lost <= 1'b0;
    end
    // read-to-clear on RXDATA
    if (apb_read && paddr == ADDR_RXDATA)
        sticky_rx_valid <= 1'b0;
end

assign irq = irq_en & sticky_done;

// ----- reads -----
always @(*) begin
    prdata = 32'd0;
    case (paddr)
        ADDR_CTRL:  prdata = {27'd0, cpha, cpol, role, mode, enable};
        ADDR_CLKDIV: prdata = {16'd0, clkdiv};
        ADDR_TXDATA: prdata = {24'd0, tx_data};
        ADDR_RXDATA: prdata = {24'd0, rx_data_reg};
        ADDR_ADDR:   prdata = {25'd0, i2c_addr};
        ADDR_CMD:    prdata = {30'd0, rw, 1'b0};
        ADDR_STATUS: prdata = {27'd0, sticky_rx_valid, sticky_arb_lost,
                                sticky_ack_err, sticky_done, busy_i};
        ADDR_IRQEN:  prdata = {31'd0, irq_en};
        default:     prdata = 32'd0;
    endcase
end
endmodule

```

A.7 spi_i2c_top.v

```

//=====
// spi_i2c_top.v
// Top-level dual-mode SPI/I2C peripheral controller.

```

```

//
// - APB-lite register interface (spi_i2c_regs) selects mode (SPI/I2C) and
//   role (MASTER/SLAVE) at runtime; only one engine is active at a time.
// - Each of the four protocol engines sits behind its own clock-gate cell;
//   only the engine matching {mode,role} (and CTRL.enable) gets a clock,
//   the other three are frozen - the low-power technique from the project
//   brief, applied at the block level.
// - SPI pins are a shared inout bus muxed by role; I2C pins are a true
//   open-drain inout bus (SCL/SDA) shared by master and slave logic and
//   released (Z) whenever neither engine is driving, matching real I2C
//   wired-AND behavior with an external pull-up.
//=====
module spi_i2c_top (
    input wire      clk,
    input wire      rst_n,
    input wire      test_en,    // scan/DFT clock-gate bypass

    // APB-lite
    input wire      psel,
    input wire      penable,
    input wire      pwrite,
    input wire [7:0] paddr,
    input wire [31:0] pwrdata,
    output wire [31:0] prdata,
    output wire      pready,
    output wire      irq,

    // SPI shared pins
    inout wire      sclk_io,
    inout wire      mosi_io,
    inout wire      miso_io,
    inout wire      cs_n_io,

    // I2C shared pins (open-drain)
    inout wire      scl_io,
    inout wire      sda_io
);

    localparam MODE_SPI = 1'b0;
    localparam MODE_I2C = 1'b1;
    localparam ROLE_MASTER = 1'b0;
    localparam ROLE_SLAVE = 1'b1;

    // -----
    // register file
    // -----
    wire      enable, mode, role, cpol, cpha, rw, start_pulse;
    wire [15:0] clkdiv;
    wire [7:0] tx_data;
    wire [6:0] i2c_addr;

    wire      busy_i, done_i, rx_valid_i, ack_err_i, arb_lost_i;
    wire [7:0] rx_data_i;

    spi_i2c_regs u_regs (
        .clk      (clk),
        .rst_n    (rst_n),
        .psel     (psel),
        .penable  (penable),
        .pwrite   (pwrite),
        .paddr    (paddr),
        .pwrdata  (pwrdata),
        .prdata   (prdata),
        .pready   (pready),
        .enable   (enable),
        .mode     (mode),
        .role     (role),
        .cpol     (cpol),
        .cpha     (cpha),
        .clkdiv   (clkdiv),
        .tx_data  (tx_data),

```

```

        .i2c_addr    (i2c_addr),
        .rw          (rw),
        .start_pulse(start_pulse),
        .busy_i      (busy_i),
        .done_i      (done_i),
        .rx_data_i   (rx_data_i),
        .rx_valid_i  (rx_valid_i),
        .ack_err_i   (ack_err_i),
        .arb_lost_i  (arb_lost_i),
        .irq         (irq)
    );

// -----
// per-engine clock gating: only master engines are gated. The slave
// engines contain input synchronizers that continuously track the
// external, asynchronous bus; if their clock is frozen while
// deselected, that synchronizer holds a stale bus level and can
// mis-detect the very first edge of the next transaction after being
// re-enabled. Masters don't have this problem - they drive the bus
// off their own internal timeline - so only they get gated.
// -----
wire sel_spi_m = enable && (mode == MODE_SPI) && (role == ROLE_MASTER);
wire sel_spi_s = enable && (mode == MODE_SPI) && (role == ROLE_SLAVE);
wire sel_i2c_m = enable && (mode == MODE_I2C) && (role == ROLE_MASTER);
wire sel_i2c_s = enable && (mode == MODE_I2C) && (role == ROLE_SLAVE);

wire clk_spi_m, clk_i2c_m;

clk_gate u_cg_spi_m (.clk_in(clk), .enable(sel_spi_m), .test_en(test_en), .clk_out(clk_spi_m));
clk_gate u_cg_i2c_m (.clk_in(clk), .enable(sel_i2c_m), .test_en(test_en), .clk_out(clk_i2c_m));

// -----
// SPI master
// -----
wire spi_m_sclk, spi_m_mosi, spi_m_cs_n;
wire spi_m_miso_in = miso_io;
wire [7:0] spi_m_rx_data;
wire spi_m_busy, spi_m_done;

spi_master #(.DW(8)) u_spi_master (
    .clk      (clk_spi_m),
    .rst_n    (rst_n),
    .en       (sel_spi_m),
    .start    (start_pulse),
    .cpol     (cpol),
    .cpha     (cpha),
    .clkdiv   (clkdiv),
    .tx_data  (tx_data),
    .rx_data  (spi_m_rx_data),
    .busy     (spi_m_busy),
    .done     (spi_m_done),
    .sclk     (spi_m_sclk),
    .mosi     (spi_m_mosi),
    .miso     (spi_m_miso_in),
    .cs_n     (spi_m_cs_n)
);

// -----
// SPI slave (always-on clock - see clock-gating note above)
// -----
wire spi_s_miso_out;
wire [7:0] spi_s_rx_data;
wire spi_s_busy, spi_s_rx_valid;

spi_slave #(.DW(8)) u_spi_slave (
    .clk      (clk),
    .rst_n    (rst_n),
    .en       (sel_spi_s),
    .cpol     (cpol),
    .cpha     (cpha),
    .sclk_in  (sclk_io),

```

```

        .mosi_in  (mosi_io),
        .cs_n_in  (cs_n_io),
        .miso_out (spi_s_miso_out),
        .tx_data  (tx_data),
        .rx_data  (spi_s_rx_data),
        .rx_valid (spi_s_rx_valid),
        .busy     (spi_s_busy)
    );

// SPI pin muxing: master drives sclk/mosi/cs_n and reads miso;
// slave reads sclk/mosi/cs_n and drives miso. Undriven lines float.
assign sclk_io = (sel_spi_m) ? spi_m_sclk : 1'bz;
assign mosi_io = (sel_spi_m) ? spi_m_mosi : 1'bz;
assign cs_n_io = (sel_spi_m) ? spi_m_cs_n : 1'bz;
assign miso_io = (sel_spi_s) ? spi_s_miso_out : 1'bz;

// -----
// I2C master
// -----
wire i2c_m_scl_oe, i2c_m_sda_oe;
wire [7:0] i2c_m_rd_data;
wire i2c_m_busy, i2c_m_done, i2c_m_ack_err, i2c_m_arb_lost;

i2c_master #(.AW(7)) u_i2c_master (
    .clk      (clk_i2c_m),
    .rst_n    (rst_n),
    .en       (sel_i2c_m),
    .start    (start_pulse),
    .clkdiv   (clkdiv),
    .slv_addr (i2c_addr),
    .rw       (rw),
    .wr_data  (tx_data),
    .rd_data  (i2c_m_rd_data),
    .busy     (i2c_m_busy),
    .done     (i2c_m_done),
    .ack_err  (i2c_m_ack_err),
    .arb_lost (i2c_m_arb_lost),
    .scl_oe   (i2c_m_scl_oe),
    .scl_in   (scl_io),
    .sda_oe   (i2c_m_sda_oe),
    .sda_in   (sda_io)
);

// -----
// I2C slave (always-on clock - see clock-gating note above)
// -----
wire i2c_s_sda_oe;
wire [7:0] i2c_s_rx_data;
wire i2c_s_busy, i2c_s_rx_valid, i2c_s_addr_match;

i2c_slave #(.AW(7)) u_i2c_slave (
    .clk      (clk),
    .rst_n    (rst_n),
    .en       (sel_i2c_s),
    .own_addr (i2c_addr),
    .tx_data  (tx_data),
    .rx_data  (i2c_s_rx_data),
    .rx_valid (i2c_s_rx_valid),
    .busy     (i2c_s_busy),
    .addr_match (i2c_s_addr_match),
    .scl_in   (scl_io),
    .sda_oe   (i2c_s_sda_oe),
    .sda_in   (sda_io)
);

// I2C open-drain pin muxing: pull low if any active engine asserts oe,
// otherwise release (external pull-up wins).
assign scl_io = (sel_i2c_m && i2c_m_scl_oe) ? 1'b0 : 1'bz;
assign sda_io = ((sel_i2c_m && i2c_m_sda_oe) || (sel_i2c_s && i2c_s_sda_oe)) ? 1'b0 : 1'bz;

// -----

```

```

// status/data mux back to the register file
// -----
assign busy_i      = sel_spi_m ? spi_m_busy :
                    sel_spi_s ? spi_s_busy :
                    sel_i2c_m ? i2c_m_busy :
                    sel_i2c_s ? i2c_s_busy : 1'b0;

assign done_i      = sel_spi_m ? spi_m_done : sel_i2c_m ? i2c_m_done : 1'b0;
assign rx_valid_i  = sel_spi_s ? spi_s_rx_valid :
                    sel_i2c_s ? i2c_s_rx_valid :
                    (sel_spi_m && spi_m_done) ? 1'b1 :
                    (sel_i2c_m && i2c_m_done && rw) ? 1'b1 : 1'b0;

assign rx_data_i   = sel_spi_m ? spi_m_rx_data :
                    sel_spi_s ? spi_s_rx_data :
                    sel_i2c_m ? i2c_m_rd_data :
                    sel_i2c_s ? i2c_s_rx_data : 8'd0;

assign ack_err_i   = sel_i2c_m ? i2c_m_ack_err : 1'b0;
assign arb_lost_i  = sel_i2c_m ? i2c_m_arb_lost : 1'b0;

endmodule

```

A.8 tb/spi_i2c_pkg.sv

```

//=====
// spi_i2c_pkg.sv
// Shared parameters/types for the SPI/I2C controller testbench.
// Mirrors the register map defined in rtl/spi_i2c_regs.v.
//=====
package spi_i2c_pkg;

    localparam bit [7:0] ADDR_CTRL    = 8'h00;
    localparam bit [7:0] ADDR_CLKDIV = 8'h04;
    localparam bit [7:0] ADDR_TXDATA  = 8'h08;
    localparam bit [7:0] ADDR_RXDATA  = 8'h0C;
    localparam bit [7:0] ADDR_ADDR    = 8'h10;
    localparam bit [7:0] ADDR_CMD     = 8'h14;
    localparam bit [7:0] ADDR_STATUS  = 8'h18;
    localparam bit [7:0] ADDR_IRQEN   = 8'h1C;

    localparam bit MODE_SPI = 1'b0;
    localparam bit MODE_I2C = 1'b1;
    localparam bit ROLE_MASTER = 1'b0;
    localparam bit ROLE_SLAVE  = 1'b1;

    // CTRL[4:0] = {cpha,cpol,role,mode,enable}
    function automatic logic [31:0] ctrl_word(bit en, bit mode, bit role,
                                              bit cpol, bit cpha);
        ctrl_word = {27'd0, cpha, cpol, role, mode, en};
    endfunction

    // CMD[1:0] = {rw,start}
    function automatic logic [31:0] cmd_word(bit start, bit rw);
        cmd_word = {30'd0, rw, start};
    endfunction

    typedef struct packed {
        logic busy;
        logic done;
        logic ack_err;
        logic arb_lost;
        logic rx_valid;
    } status_t;

    function automatic status_t decode_status(logic [31:0] w);
        decode_status.busy      = w[0];
        decode_status.done      = w[1];
        decode_status.ack_err    = w[2];
    endfunction

```

```

        decode_status.arb_lost = w[3];
        decode_status.rx_valid = w[4];
    endfunction

```

```

endpackage

```

A.9 tb/spi_i2c_assertions.sv

```

//=====
// spi_i2c_assertions.sv
// Protocol-compliance checkers, bound into the RTL so they run "for free"
// in any simulation that elaborates spi_i2c_top - no testbench wiring
// needed beyond compiling this file alongside the DUT.
//=====

// -----
// Top-level checks: engine mutual exclusion (the clock-gating mux must only
// ever enable exactly one engine at a time) and basic SPI idle behavior.
// -----
module spi_i2c_top_checker (
    input logic clk,
    input logic rst_n,
    input logic sel_spi_m,
    input logic sel_spi_s,
    input logic sel_i2c_m,
    input logic sel_i2c_s,
    input logic cs_n_io,
    input logic sclk_io
);

    // At most one engine selected at any time -> at most one ungated clock.
    property p_mutex_engine_sel;
        @(posedge clk) disable iff (!rst_n)
            $onehot0({sel_spi_m, sel_spi_s, sel_i2c_m, sel_i2c_s});
    endproperty
    a_mutex_engine_sel: assert property (p_mutex_engine_sel)
        else $error("[ASSERT] more than one protocol engine selected simultaneously");

    // SCLK must be static while CS_N is deasserted (bus idle) in SPI master mode.
    property p_sclk_static_when_idle;
        @(posedge clk) disable iff (!rst_n)
            (sel_spi_m && cs_n_io) |-> $stable(sclk_io);
    endproperty
    a_sclk_static_when_idle: assert property (p_sclk_static_when_idle)
        else $error("[ASSERT] SCLK toggled while CS_N deasserted (SPI master idle)");

endmodule

bind spi_i2c_top spi_i2c_top_checker u_top_checker (
    .clk      (clk),
    .rst_n    (rst_n),
    .sel_spi_m (sel_spi_m),
    .sel_spi_s (sel_spi_s),
    .sel_i2c_m (sel_i2c_m),
    .sel_i2c_s (sel_i2c_s),
    .cs_n_io   (cs_n_io),
    .sclk_io   (sclk_io)
);

// -----
// I2C bus-level checks, bound into i2c_slave since it already computes
// synchronized scl_s/sda_s and edge/condition detects on the shared bus.
// -----
module i2c_slave_checker (
    input logic clk,
    input logic rst_n,
    input logic scl_s,
    input logic sda_s,
    input logic scl_prev,

```

```

input logic sda_prev,
input logic start_cond,
input logic stop_cond,
input logic busy
);

// Fundamental I2C rule: SDA may only change while SCL is high if that
// change IS a START or STOP condition. Any other SDA transition while
// SCL is steady-high is a protocol violation.
property p_sda_stable_when_scl_high;
  @(posedge clk) disable iff (!rst_n)
    (scl_prev && scl_s && (sda_prev != sda_s)) |-> (start_cond || stop_cond);
endproperty
a_sda_stable_when_scl_high: assert property (p_sda_stable_when_scl_high)
  else $error("[ASSERT] illegal SDA change while SCL held high (not START/STOP)");

// A STOP condition should only be observed while a transaction is (or
// was) in progress in this simple single-master-at-a-time testbench;
// catches a monitor mis-wiring more than a real protocol bug.
property p_stop_implies_was_active_or_idle;
  @(posedge clk) disable iff (!rst_n)
    stop_cond |-> 1'b1; // structural placeholder: always legal, kept for
                        // symmetry / easy extension to multi-master checks
endproperty
a_stop_sanity: assert property (p_stop_implies_was_active_or_idle);

endmodule

bind i2c_slave i2c_slave_checker u_i2c_slave_checker (
  .clk      (clk),
  .rst_n    (rst_n),
  .scl_s    (scl_s),
  .sda_s    (sda_s),
  .scl_prev (scl_prev),
  .sda_prev (sda_prev),
  .start_cond (start_cond),
  .stop_cond (stop_cond),
  .busy      (busy)
);

// -----
// SPI slave checks: MOSI/CS/SCLK must be glitch-free once synchronized, and
// bit_cnt must never be sampled X/unknown mid-transfer (catches reset/CDC
// bugs early, before they show up as a wrong byte 200 cycles later).
// -----
module spi_slave_checker (
  input logic clk,
  input logic rst_n,
  input logic busy,
  input logic [3:0] bit_cnt
);

property p_no_unknown_bitcnt_when_busy;
  @(posedge clk) disable iff (!rst_n)
    busy |-> !$isunknown(bit_cnt);
endproperty
a_no_unknown_bitcnt: assert property (p_no_unknown_bitcnt_when_busy)
  else $error("[ASSERT] spi_slave.bit_cnt is X/Z while busy");

endmodule

bind spi_slave spi_slave_checker u_spi_slave_checker (
  .clk      (clk),
  .rst_n    (rst_n),
  .busy     (busy),
  .bit_cnt  (bit_cnt)
);

```

A.10 tb/spi_i2c_tb_top.sv

```
//=====
// spi_i2c_tb_top.sv
// Top-level testbench. Two spi_i2c_top instances share the SPI bus and the
// I2C bus, one configured as MASTER and the other as SLAVE, so the DUT
// verifies itself against itself in both protocols and both directions.
//
// Run (Icarus Verilog):
// iverilog -g2012 -o sim.out -I ../rtl \
//      ../rtl/*.v spi_i2c_pkg.sv spi_i2c_assertions.sv spi_i2c_tb_top.sv
// vvp sim.out
//
// Run (Verilator, lint + sim):
// verilator --binary -Wall --timing -sv \
//      ../rtl/*.v spi_i2c_pkg.sv spi_i2c_assertions.sv spi_i2c_tb_top.sv \
//      --top-module spi_i2c_tb_top -o sim.out
//      ./obj_dir/sim.out
//=====
`timescale 1ns/1ps

import spi_i2c_pkg::*;

interface apb_if (input logic clk);
    logic        psel;
    logic        penable;
    logic        pwrite;
    logic [7:0]  paddr;
    logic [31:0] pwdata;
    logic [31:0] prdata;
    logic        pready;

    task automatic write(logic [7:0] addr, logic [31:0] data);
        @(posedge clk);
        psel    <= 1'b1;
        pwrite  <= 1'b1;
        paddr   <= addr;
        pwdata  <= data;
        penable <= 1'b0;
        @(posedge clk);
        penable <= 1'b1;
        @(posedge clk);
        psel    <= 1'b0;
        penable <= 1'b0;
        pwrite  <= 1'b0;
    endtask

    task automatic read(logic [7:0] addr, output logic [31:0] data);
        @(posedge clk);
        psel    <= 1'b1;
        pwrite  <= 1'b0;
        paddr   <= addr;
        penable <= 1'b0;
        @(posedge clk);
        penable <= 1'b1;
        @(posedge clk);
        data    = prdata;
        psel    <= 1'b0;
        penable <= 1'b0;
    endtask
endinterface

module spi_i2c_tb_top;

    // ----- clock / reset -----
    logic clk = 0;
    logic rst_n = 0;
    always #5 clk = ~clk; // 100 MHz

    initial begin
        rst_n = 0;
    end
endmodule
```

```

        repeat (5) @(posedge clk);
        rst_n = 1;
end

// ----- shared buses -----
wire sclk_bus, mosi_bus, miso_bus, cs_n_bus;
wire scl_bus, sda_bus;
pullup(scl_bus);
pullup(sda_bus);

// ----- APB interfaces -----
apb_if apb_a(clk);
apb_if apb_b(clk);

// ----- DUTs -----
// dut_a: configured as the MASTER side in each test
// dut_b: configured as the SLAVE side in each test
spi_i2c_top dut_a (
    .clk(clk), .rst_n(rst_n), .test_en(1'b0),
    .psel(apb_a.psel), .penable(apb_a.penable), .pwrite(apb_a.pwrite),
    .paddr(apb_a.paddr), .pdata(apb_a.pdata), .prdata(apb_a.prdata),
    .pready(apb_a.pready), .irq(),
    .sclk_io(sclk_bus), .mosi_io(mosi_bus), .miso_io(miso_bus), .cs_n_io(cs_n_bus),
    .scl_io(scl_bus), .sda_io(sda_bus)
);

spi_i2c_top dut_b (
    .clk(clk), .rst_n(rst_n), .test_en(1'b0),
    .psel(apb_b.psel), .penable(apb_b.penable), .pwrite(apb_b.pwrite),
    .paddr(apb_b.paddr), .pdata(apb_b.pdata), .prdata(apb_b.prdata),
    .pready(apb_b.pready), .irq(),
    .sclk_io(sclk_bus), .mosi_io(mosi_bus), .miso_io(miso_bus), .cs_n_io(cs_n_bus),
    .scl_io(scl_bus), .sda_io(sda_bus)
);

// ----- scoreboard bookkeeping -----
int pass_cnt = 0;
int fail_cnt = 0;

task automatic check_equal(string what, logic [31:0] act, logic [31:0] exp);
    if (act == exp) begin
        pass_cnt++;
        $display("[PASS] %-40s got=0x%0h exp=0x%0h", what, act, exp);
    end else begin
        fail_cnt++;
        $error("[FAIL] %-40s got=0x%0h exp=0x%0h", what, act, exp);
    end
endtask

task automatic check_true(string what, logic act);
    if (act == 1'b1) begin
        pass_cnt++;
        $display("[PASS] %-40s", what);
    end else begin
        fail_cnt++;
        $error("[FAIL] %-40s (expected 1'b1, got %0b)", what, act);
    end
endtask

// poll STATUS.busy on apb_a until it clears, or timeout
task automatic wait_done(string tag);
    logic [31:0] st;
    int timeout = 2000;
    st = 32'h1;
    while (st[0] && timeout > 0) begin
        apb_a.read(ADDR_STATUS, st);
        timeout--;
        repeat (5) @(posedge clk);
    end
    if (timeout == 0) begin
        fail_cnt++;
    end
endtask

```

```

        $error("[FAIL] %s: timed out waiting for busy to clear", tag);
    end
endtask

// ----- functional coverage -----
// NOTE: SystemVerilog `covergroup`/`coverpoint`/`cross` are a
// commercial-simulator feature (VCS/Quest/Xcelium) that Icarus
// Verilog does not implement. To keep this testbench runnable on the
// free open-source toolchain, coverage is tracked manually here with
// plain bit vectors instead - same intent (did we exercise every
// interesting case?), portable syntax. If you have access to a
// simulator with real covergroup support, swap this block back for
// the covergroup version described in docs/README.md.

// bit index = {cpol, cpha} -> all 4 SPI modes
reg [3:0] spi_mode_hit = 4'b0000;
// bit index = {rw, ack_err} -> I2C write-ack, write-nack, read-ack, read-nack
reg [3:0] i2c_txn_hit = 4'b0000;
// bit0=zero byte seen, bit1=0xFF seen, bit2=some mid-range byte seen
reg [2:0] byte_bin_hit = 3'b000;

task automatic mark_spi_mode(bit cpol, bit cpha);
    spi_mode_hit[{cpol, cpha}] = 1'b1;
endtask

task automatic mark_i2c_txn(bit rw, bit ack_err);
    i2c_txn_hit[{rw, ack_err}] = 1'b1;
endtask

function automatic int popcount4(logic [3:0] v);
    popcount4 = v[0] + v[1] + v[2] + v[3];
endfunction

function automatic int popcount3(logic [2:0] v);
    popcount3 = v[0] + v[1] + v[2];
endfunction

task automatic mark_byte(logic [7:0] b);
    if (b == 8'h00)    byte_bin_hit[0] = 1'b1;
    else if (b == 8'hFF) byte_bin_hit[1] = 1'b1;
    else              byte_bin_hit[2] = 1'b1;
endtask

// ----- SPI directed test -----
task automatic run_spi_test(bit cpol, bit cpha, logic [7:0] m_tx, logic [7:0] s_tx);
    logic [31:0] rd;

    $display("\n---- SPI test: CPOL=%0b CPHA=%0b m_tx=0x%0h s_tx=0x%0h ----",
        cpol, cpha, m_tx, s_tx);

    apb_a.write(ADDR_CTRL, ctrl_word(1'b0, MODE_SPI, ROLE_MASTER, cpol, cpha));
    apb_b.write(ADDR_CTRL, ctrl_word(1'b0, MODE_SPI, ROLE_SLAVE, cpol, cpha));
    apb_a.write(ADDR_CLKDIV, 32'd4);
    apb_b.write(ADDR_CLKDIV, 32'd4);
    apb_a.write(ADDR_TXDATA, m_tx);
    apb_b.write(ADDR_TXDATA, s_tx);

    apb_a.write(ADDR_CTRL, ctrl_word(1'b1, MODE_SPI, ROLE_MASTER, cpol, cpha));
    apb_b.write(ADDR_CTRL, ctrl_word(1'b1, MODE_SPI, ROLE_SLAVE, cpol, cpha));
    repeat (5) @(posedge clk);

    apb_a.write(ADDR_CMD, cmd_word(1'b1, 1'b0)); // start pulse
    wait_done("spi_master");
    repeat (10) @(posedge clk);

    apb_a.read(ADDR_RXDATA, rd);
    check_equal($sformatf("SPI master captured slave byte (cpol=%0b cpha=%0b)", cpol, cpha),
        rd[7:0], s_tx);

    apb_b.read(ADDR_RXDATA, rd);
    check_equal($sformatf("SPI slave captured master byte (cpol=%0b cpha=%0b)", cpol, cpha),
        rd[7:0], m_tx);

```

```

// disable both before next test to force the top-level mux back to
// "nothing selected" and exercise the clock-gating idle path
apb_a.write(ADDR_CTRL, 32'h0);
apb_b.write(ADDR_CTRL, 32'h0);

mark_spi_mode(cpol, cpha);
mark_byte(m_tx);
mark_byte(s_tx);
endtask

// ----- I2C directed test -----
task automatic run_i2c_write(logic [6:0] addr, logic [6:0] slv_own_addr, logic [7:0] data);
    logic [31:0] rd;
    bit expect_ack = (addr == slv_own_addr);

    $display("\n---- I2C write test: taddr=0x%0h slv_addr=0x%0h data=0x%0h ----",
        addr, slv_own_addr, data);

    apb_a.write(ADDR_CTRL, ctrl_word(1'b0, MODE_I2C, ROLE_MASTER, 1'b0, 1'b0));
    apb_b.write(ADDR_CTRL, ctrl_word(1'b0, MODE_I2C, ROLE_SLAVE, 1'b0, 1'b0));
    apb_a.write(ADDR_CLKDIV, 32'd8);
    apb_b.write(ADDR_ADDR, slv_own_addr);
    apb_a.write(ADDR_ADDR, addr);
    apb_a.write(ADDR_TXDATA, data);

    apb_a.write(ADDR_CTRL, ctrl_word(1'b1, MODE_I2C, ROLE_MASTER, 1'b0, 1'b0));
    apb_b.write(ADDR_CTRL, ctrl_word(1'b1, MODE_I2C, ROLE_SLAVE, 1'b0, 1'b0));
    apb_a.write(ADDR_STATUS, 32'h0000000E); // W1C: clear done/ack_err/arb_lost
    repeat (5) @(posedge clk);

    apb_a.write(ADDR_CMD, cmd_word(1'b1, 1'b0)); // start pulse, rw=0 (write)
    wait_done("i2c_master_write");
    repeat (10) @(posedge clk);

    apb_a.read(ADDR_STATUS, rd);
    if (expect_ack) begin
        check_equal("I2C write: address ACKed (ack_err==0)", rd[2], 1'b0);
        apb_b.read(ADDR_STATUS, rd);
        check_equal("I2C write: slave rx_valid set", rd[4], 1'b1);
        apb_b.read(ADDR_RXDATA, rd);
        check_equal("I2C write: slave received correct byte", rd[7:0], data);
    end else begin
        check_equal("I2C write: mismatched address NACKed (ack_err==1)", rd[2], 1'b1);
    end

    apb_a.write(ADDR_CTRL, 32'h0);
    apb_b.write(ADDR_CTRL, 32'h0);

    mark_i2c_txn(1'b0, !expect_ack);
endtask

task automatic run_i2c_read(logic [6:0] addr, logic [6:0] slv_own_addr, logic [7:0] slv_data);
    logic [31:0] rd;

    $display("\n---- I2C read test: taddr=0x%0h slv_addr=0x%0h slv_data=0x%0h ----",
        addr, slv_own_addr, slv_data);

    apb_a.write(ADDR_CTRL, ctrl_word(1'b0, MODE_I2C, ROLE_MASTER, 1'b0, 1'b0));
    apb_b.write(ADDR_CTRL, ctrl_word(1'b0, MODE_I2C, ROLE_SLAVE, 1'b0, 1'b0));
    apb_a.write(ADDR_CLKDIV, 32'd8);
    apb_b.write(ADDR_ADDR, slv_own_addr);
    apb_a.write(ADDR_ADDR, addr);
    apb_b.write(ADDR_TXDATA, slv_data); // byte the slave will return

    apb_a.write(ADDR_CTRL, ctrl_word(1'b1, MODE_I2C, ROLE_MASTER, 1'b0, 1'b0));
    apb_b.write(ADDR_CTRL, ctrl_word(1'b1, MODE_I2C, ROLE_SLAVE, 1'b0, 1'b0));
    apb_a.write(ADDR_STATUS, 32'h0000000E); // W1C: clear done/ack_err/arb_lost
    repeat (5) @(posedge clk);

    apb_a.write(ADDR_CMD, cmd_word(1'b1, 1'b1)); // start pulse, rw=1 (read)

```

```

wait_done("i2c_master_read");
repeat (10) @(posedge clk);

apb_a.read(ADDR_STATUS, rd);
check_equal("I2C read: address ACKed (ack_err==0)", rd[2], 1'b0);
apb_a.read(ADDR_RXDATA, rd);
check_equal("I2C read: master received correct byte", rd[7:0], slv_data);

apb_a.write(ADDR_CTRL, 32'h0);
apb_b.write(ADDR_CTRL, 32'h0);

mark_i2c_txn(1'b1, 1'b0);
mark_byte(slv_data);
endtask

// ----- test sequence -----
initial begin
    @(posedge rst_n);
    repeat (10) @(posedge clk);

    // SPI: sweep all 4 CPOL/CPHA modes with edge-case + mid-range bytes
    run_spi_test(1'b0, 1'b0, 8'hA5, 8'h5A);
    run_spi_test(1'b0, 1'b1, 8'h00, 8'hFF);
    run_spi_test(1'b1, 1'b0, 8'hFF, 8'h00);
    run_spi_test(1'b1, 1'b1, 8'h3C, 8'hC3);

    // I2C: matching address write, matching address read, mismatched address
    run_i2c_write(7'h50, 7'h50, 8'h42);
    run_i2c_read (7'h50, 7'h50, 8'h99);
    run_i2c_write(7'h51, 7'h50, 8'hDE); // address mismatch -> expect NACK

    repeat (20) @(posedge clk);

    $display("\n===== COVERAGE SUMMARY =====");
    $display("SPI mode coverage      : %0d/4 modes   (%0.1f%%)",
        popcount4(spi_mode_hit), popcount4(spi_mode_hit) * 100.0 / 4.0);
    $display("I2C txn coverage          : %0d/4 combos  (%0.1f%%)",
        popcount4(i2c_txn_hit), popcount4(i2c_txn_hit) * 100.0 / 4.0);
    $display("Byte value coverage       : %0d/3 bins   (%0.1f%%)",
        popcount3(byte_bin_hit), popcount3(byte_bin_hit) * 100.0 / 3.0);

    $display("\n===== REGRESSION SUMMARY =====");
    $display("PASS: %0d  FAIL: %0d", pass_cnt, fail_cnt);
    if (fail_cnt == 0)
        $display("REGRESSION: PASS");
    else
        $display("REGRESSION: FAIL");

    $finish;
end

// safety timeout in case a wait_done loop or DUT hang isn't caught
initial begin
    #200000;
    $display("REGRESSION: FAIL (global timeout)");
    $finish;
end

endmodule

```

A.11 scripts/regression.py

```

#!/usr/bin/env python3
"""
regression.py
Scripted regression runner for the SPI/I2C controller project.

What it does:
1. Compiles + simulates the SystemVerilog testbench (Icarus Verilog by

```

default, Verilator optionally) and scans the log for the "REGRESSION: PASS/FAIL" marker plus any [FAIL]/assertion errors.

2. Optionally runs a Yosys lint pass (`check`) on the RTL on its own, independent of whether a liberty file is available, to catch latches-where-not-expected, multi-driven nets, width mismatches, etc.
3. Optionally runs the full Yosys synthesis script if a liberty path is given, and reports the resulting cell/area summary.
4. Prints a single pass/fail summary and exits non-zero on any failure, so this can be dropped straight into a CI job.

Usage:

```
python3 regression.py                # sim only, iverilog
python3 regression.py --sim verilator # sim only, verilator
python3 regression.py --lint         # + yosys lint
python3 regression.py --synth --liberty x.lib # + yosys synthesis
python3 regression.py --all --liberty x.lib  # everything
"""

import argparse
import re
import subprocess
import sys
import time
from pathlib import Path

ROOT      = Path(__file__).resolve().parent.parent
RTL_DIR   = ROOT / "rtl"
TB_DIR    = ROOT / "tb"
SYN_DIR   = ROOT / "syn"
BUILD     = ROOT / "build"

RTL_FILES = sorted(RTL_DIR.glob("*.v"))
# Note: spi_i2c_assertions.sv (SVA property/assert property checkers) is
# deliberately excluded here - Icarus Verilog doesn't support concurrent
# assertions. If you're running with a simulator that does (Verilator with
# --assert, or a commercial simulator), add it back to this list.
TB_FILES = [
    TB_DIR / "spi_i2c_pkg.sv",
    TB_DIR / "spi_i2c_tb_top.sv",
]

PASS_MARKER = "REGRESSION: PASS"
FAIL_MARKER = "REGRESSION: FAIL"
FAIL_LINE_RE = re.compile(r"\[FAIL\]|ASSERT|\$error", re.IGNORECASE)

def run(cmd, cwd=None):
    print(f"$ {' '.join(cmd)}")
    t0 = time.time()
    proc = subprocess.run(cmd, cwd=cwd, capture_output=True, text=True)
    dt = time.time() - t0
    return proc.returncode, proc.stdout + proc.stderr, dt

def sim_iverilog():
    BUILD.mkdir(exist_ok=True)
    out_bin = BUILD / "sim.out"
    cmd = ["iverilog", "-g2012", "-o", str(out_bin)] + \
        [str(f) for f in RTL_FILES] + [str(f) for f in TB_FILES]
    rc, log, dt = run(cmd)
    if rc != 0:
        return False, f"[compile failed, {dt:.1f}s]\n{log}"

    rc, log, dt = run(["vvp", str(out_bin)])
    return rc == 0, f"[sim ran in {dt:.1f}s]\n{log}"

def sim_verilator():
    BUILD.mkdir(exist_ok=True)
    cmd = [
        "verilator", "--binary", "-Wall", "--timing", "-sv",
```

```

        "-Wno-DECLFILENAME", "-Wno-UNUSED SIGNAL",
        "--top-module", "spi_i2c_tb_top",
        "-Mdir", str(BUILD / "obj_dir"),
        "-o", "sim.out",
    ] + [str(f) for f in RTL_FILES] + [str(f) for f in TB_FILES]
    rc, log, dt = run(cmd)
    if rc != 0:
        return False, f"[verilate failed, {dt:.1f}s]\n{log}"

    sim_bin = BUILD / "obj_dir" / "sim.out"
    rc, log, dt = run([str(sim_bin)])
    return rc == 0, f"[sim ran in {dt:.1f}s]\n{log}"

def yosys_lint():
    script = f"""
    read_verilog -sv {' '.join(str(f) for f in RTL_FILES)}
    hierarchy -check -top spi_i2c_top
    check -mapped
    """
    rc, log, dt = run(["yosys", "-p", script])
    return rc == 0, f"[lint ran in {dt:.1f}s]\n{log}"

def yosys_synth(liberty):
    cmd = ["yosys", "-c", str(SYN_DIR / "yosys_synth.tcl")]
    if liberty:
        cmd += ["-", liberty]
    rc, log, dt = run(cmd, cwd=ROOT)
    return rc == 0, f"[synth ran in {dt:.1f}s]\n{log}"

def evaluate_sim_log(log: str) -> tuple[bool, list[str]]:
    """Return (passed, list_of_failure_lines) based on markers + error scan."""
    failures = [ln for ln in log.splitlines() if FAIL_LINE_RE.search(ln)]
    if PASS_MARKER in log and FAIL_MARKER not in log and not failures:
        return True, []
    return False, failures

def main():
    ap = argparse.ArgumentParser(description=__doc__)
    ap.add_argument("--sim", choices=["iverilog", "verilator", "none"], default="iverilog")
    ap.add_argument("--lint", action="store_true", help="run yosys `check` lint pass")
    ap.add_argument("--synth", action="store_true", help="run full yosys synthesis")
    ap.add_argument("--liberty", default="", help="liberty file for --synth (optional)")
    ap.add_argument("--all", action="store_true", help="run sim + lint + synth")
    args = ap.parse_args()

    if args.all:
        args.lint = True
        args.synth = True

    overall_ok = True
    sections = []

    if args.sim != "none":
        print(f"\n== Simulation ({args.sim}) ==")
        if args.sim == "iverilog":
            ran_ok, log = sim_iverilog()
        else:
            ran_ok, log = sim_verilator()

        print(log)
        sim_ok, failures = evaluate_sim_log(log) if ran_ok else (False, ["tool failed to run"])
        overall_ok &= sim_ok
        sections.append(("simulation", sim_ok, failures))

    if args.lint:
        print("\n== Yosys lint ==")
        ok, log = yosys_lint()

```

```

    print(log)
    overall_ok &= ok
    sections.append(("yosys lint", ok, [] if ok else ["yosys check reported errors"]))

if args.synth:
    print("\n== Yosys synthesis ==")
    ok, log = yosys_synth(args.liberty)
    print(log)
    overall_ok &= ok
    sections.append(("yosys synth", ok, [] if ok else ["synthesis failed"]))

print("\n===== REGRESSION REPORT =====")
for name, ok, failures in sections:
    status = "PASS" if ok else "FAIL"
    print(f" [{status}] {name}")
    for f in failures[:10]:
        print(f"      {f}")
    if len(failures) > 10:
        print(f"      ... and {len(failures) - 10} more")

print("\nOVERALL:", "PASS" if overall_ok else "FAIL")
sys.exit(0 if overall_ok else 1)

if __name__ == "__main__":
    main()

```